Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №3

по дисциплине «Серверное программирование»

Тема: оптимизация запросов и индексы

Преподаватель _____ _____ _____
подпись, дата | инициалы, фамилия

Студент _____ _____ Mohamed Amini
номер группы, зачетной книжки | подпись, дата | инициалы, фамилия

Красноярск 2026

1 Цель и постановка задачи

Цель работы – провести нагрузочное тестирование базы данных и оптимизировать типовой запрос приложения с помощью индексов. Используются схема и данные практических заданий №1 и №2.

Требования задания:

– заполнить таблицы до не менее чем 500 000 записей суммарно;

– сформулировать типовой запрос: не менее трёх таблиц и трёх условий WHERE разных типов данных;

– зафиксировать план и время выполнения до оптимизации;

– добавить индексы, перестроить их и повторить замеры;

– оформить таблицу: индексы – время – стоимость запроса.

Критерии приёмки: не менее 500 тысяч записей; EXPLAIN ANALYZE до и после; таблица сравнения производительности; выводы об эффективности индексов. Отчёт содержит SQL запроса, планы EXPLAIN, таблицу замеров и выводы.

2 Стек и запуск

Работа выполнена на СУБД PostgreSQL 17 (отдельный кластер проекта, адрес 127.0.0.1, порт 54329, база velograd) на схеме практического задания №2. База данных создана с ICU-порядком сортировки ru-RU. Запуск: npm run pg:start, затем npm start. Программа на TypeScript выполняет SQL-файлы клиентом psql; если в таблице поездок меньше 100 000 строк, сначала выполняется загрузка данных.

Таблица 1 – Скрипты работы

Файл	Назначение
sql/01_load.sql, sql/generate.sql	догрузка данных до объёма более 500 000 записей
sql/02_query.sql	типовой запрос: поездки клиентов по фамилии и имени, стоимости и дате
sql/03_indexes.sql	удаление индексов, затем шесть шагов: создание индексов и перестроение, после каждого шага – EXPLAIN ANALYZE

Файл	Назначение
sql/04_explain.sql	EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) типового запроса

3 Загрузка данных

Скрипт 01_load.sql использует тот же генератор, что и практическое задание №2 (параметры: 228 станций, 4 386 велосипедов, 47 988 клиентов, 389 984 поездки), и затем выводит число записей и размеры таблиц:

```
\set seed 0.3
\set stations 228
\set bicycles 4386
\set clients 47988
\set rentals 389984
\ir generate.sql

SELECT c.relname                                    AS table_name,
       c.reltuples::bigint                           AS rows_estimate,
       pg_size_pretty(pg_table_size(c.oid))          AS table_size,
       pg_size_pretty(pg_indexes_size(c.oid))        AS indexes_size
FROM pg_class AS c
WHERE c.relnamespace = 'public'::regnamespace AND c.relkind = 'r'
ORDER BY c.reltuples DESC;
```

После загрузки в базе 967 801 запись, из них 400 000 поездок и 343 094 платежа; критерий «не менее 500 000 записей» выполнен (рисунки 1 и 2). Статистику для планировщика обновляет команда ANALYZE в конце генератора.

```
PS ...\pz3-indexes-explain> npm start
       table_name        |  rows
-------------------------+--------
 ИТОГО                   | 967801
 district                |     10
 tariff                  |     10
 station                 |    300
 bicycle                 |   5000
 client_wallet           |  50000
 client                  |  50000
 client_favorite_station | 119387
 payment                 | 343094
 rental                  | 400000
(10 rows)
```

Рисунок 1 – Число записей в таблицах после загрузки

```
PS ...\pz3-indexes-explain> npm start
       table_name        | rows_estimate | table_size | indexes_size
-------------------------+---------------+------------+--------------
 rental                  |        400000 | 32 MB      | 8792 kB
 payment                 |        343094 | 36 MB      | 40 MB
 client_favorite_station |        119387 | 5192 kB    | 2632 kB
 client                  |         50000 | 6104 kB    | 5616 kB
 client_wallet           |         50000 | 2576 kB    | 1112 kB
 bicycle                 |          5000 | 464 kB     | 304 kB
 station                 |           300 | 72 kB      | 56 kB
 tariff                  |            10 | 16 kB      | 32 kB
 district                |            10 | 16 kB      | 32 kB
(9 rows)
```

Рисунок 2 – Размеры таблиц и индексов, созданных ограничениями (первичные ключи, UNIQUE)

4 Типовой запрос

Сценарий: найти поездки клиентов по фамилии и имени за период с фильтром по стоимости. Запрос соединяет три таблицы (rental, client, station) и содержит четыре условия трёх разных типов (таблица 2); результат сортируется по дате и ограничен ста строками.

```
SELECT c.last_name,
       c.first_name,
       s.name        AS start_station,
       r.started_at,
       r.distance_km,
       r.total_cost
FROM rental  AS r
JOIN client  AS c ON c.id = r.client_id
JOIN station AS s ON s.id = r.start_station_id
WHERE c.last_name LIKE 'Кузн%'
  AND c.first_name = 'Мария'
  AND r.total_cost BETWEEN 100 AND 600
  AND r.started_at >= timestamp '2025-01-01'
ORDER BY r.started_at DESC
LIMIT 100;
```

Таблица 2 – Условия запроса

Условие	Тип данных	Строк подходит
c.last_name LIKE 'Кузн%'	текст, префиксный поиск	962 клиента из 50 000
c.first_name = 'Мария'	текст, равенство	вместе с фамилией – 31 клиент
r.total_cost BETWEEN 100 AND 600	число (numeric)	176 557 поездок из 400 000
r.started_at >= '2025-01-01'	дата и время (timestamp)	274 493 поездки из 400 000 (68,6 %)

Запрос возвращает 71 строку; первые строки показаны на рисунке 3.

```
PS ...\pz3-indexes-explain> npm start
 last_name | first_name | start_station |     started_at      | distance_km | total_cost
-----------+------------+---------------+---------------------+-------------+------------
 Кузнецова | Мария      | Станция №216  | 2026-10-25 01:22:24 |        5.10 |     292.50
 Кузнецова | Мария      | Станция №43   | 2026-10-24 09:39:40 |        5.65 |     116.00
 Кузнецова | Мария      | Станция №215  | 2026-10-08 13:06:26 |        1.73 |     134.10
 Кузнецова | Мария      | Заречье       | 2026-09-29 21:08:19 |        1.56 |     144.00
 Кузнецова | Мария      | Станция №158  | 2026-08-31 23:07:17 |        7.89 |     371.70
 Кузнецова | Мария      | Станция №157  | 2026-08-10 14:00:54 |        5.42 |     262.80
 Кузнецова | Мария      | Станция №247  | 2026-08-08 14:47:35 |        7.98 |     172.00
…
(71 rows)
```

Рисунок 3 – Результат типового запроса (первые строки)

5 Замер до оптимизации

Замер выполняется командой EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT) с тем же запросом (файл 04_explain.sql). В базе нет индексов, кроме создаваемых ограничениями: первичных ключей и UNIQUE. Перед измерением индексы работы удаляются (первые строки скрипта 03_indexes.sql):

```
DROP INDEX IF EXISTS idx_rental_started_at;
DROP INDEX IF EXISTS idx_client_last_name;
DROP INDEX IF EXISTS idx_client_last_name_pattern;
DROP INDEX IF EXISTS idx_rental_client_id;
DROP INDEX IF EXISTS idx_rental_start_station;
```

План до оптимизации приведён на рисунке 4. Таблица поездок читается целиком параллельным последовательным просмотром (Parallel Seq Scan: 400 000 строк, из которых условиям по стоимости и дате удовлетворяют около 40 тысяч на каждый из трёх процессов), таблица клиентов – также полным просмотром (Seq Scan, 50 000 строк); результаты соединяются хэш-соединением (Hash Join), затем выполняются сортировка и ограничение LIMIT. Станция для каждой из 71 строки берётся по первичному ключу (Index Scan по station_pkey).

```
PS ...\pz3-indexes-explain> npm start   # шаг без индексов
Limit  (cost=9685.18..9690.55 rows=46 width=67) (actual time=136.647..161.637 rows=71 loops=1)
  Buffers: shared hit=5847
  ->  Gather Merge  (cost=9685.18..9690.55 rows=46 width=67) (actual time=136.644..161.629 rows=71 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=5847
        ->  Sort  (cost=8685.16..8685.22 rows=23 width=67) (actual time=47.746..47.751 rows=24 loops=3)
              Sort Key: r.started_at DESC
              Sort Method: quicksort  Memory: 29kB
              Buffers: shared hit=5847
              Worker 0:  Sort Method: quicksort  Memory: 30kB
              Worker 1:  Sort Method: quicksort  Memory: 25kB
              ->  Nested Loop  (cost=1509.43..8684.64 rows=23 width=67) (actual time=8.905..47.695 rows=24 loops=3)
                    Buffers: shared hit=5833
                    ->  Hash Join  (cost=1509.29..8680.84 rows=23 width=50) (actual time=8.886..47.595 rows=24 loops=3)
                          Hash Cond: (r.client_id = c.id)
                          Buffers: shared hit=5690
                          ->  Parallel Seq Scan on rental r  (cost=0.00..7040.67 rows=49856 width=27) (actual time=1.303..36.723
rows=40434 loops=3)
                                Filter: ((total_cost >= '100'::numeric) AND (total_cost <= '600'::numeric) AND (started_at >=
'2025-01-01 00:00:00'::timestamp without time zone))
                                Rows Removed by Filter: 92899
                                Buffers: shared hit=4124
                          ->  Hash  (cost=1509.00..1509.00 rows=23 width=31) (actual time=10.830..10.831 rows=31 loops=2)
                                Buckets: 1024  Batches: 1  Memory Usage: 11kB
                                Buffers: shared hit=1518
                                ->  Seq Scan on client c  (cost=0.00..1509.00 rows=23 width=31) (actual time=0.368..10.804 rows=31
loops=2)
                                      Filter: ((last_name ~~ 'Кузн%'::text) AND (first_name = 'Мария'::text))
                                      Rows Removed by Filter: 49969
                                      Buffers: shared hit=1518
                    ->  Index Scan using station_pkey on station s  (cost=0.15..0.17 rows=1 width=25) (actual time=0.003..0.003
rows=1 loops=71)
                          Index Cond: (id = r.start_station_id)
                          Buffers: shared hit=143
Planning:
  Buffers: shared hit=305
Planning Time: 6.781 ms
Execution Time: 161.792 ms
```

Рисунок 4 – План выполнения запроса без индексов

Одиночный первый запуск после удаления индексов оказался медленнее последующих (162 мс: запускаются параллельные процессы, прогревается кэш). Поэтому в таблице результатов указана медиана пяти повторных запусков после прогрева – установившееся значение 101,7 мс; стоимость корневого узла (cost) 9 691,5, прочитано 6 655 страниц по 8 КБ.

6 Создание и перестройка индексов

Скрипт 03_indexes.sql создаёт индексы по одному, после каждого повторяя EXPLAIN ANALYZE, а в конце перестраивает индексы командой REINDEX и обновляет статистику командой ANALYZE:

```
\ir 04_explain.sql

CREATE INDEX idx_rental_started_at ON rental (started_at);
\ir 04_explain.sql

CREATE INDEX idx_client_last_name ON client (last_name);
\ir 04_explain.sql

CREATE INDEX idx_client_last_name_pattern ON client (last_name text_pattern_ops);
\ir 04_explain.sql

CREATE INDEX idx_rental_client_id ON rental (client_id);
\ir 04_explain.sql

CREATE INDEX idx_rental_start_station ON rental (start_station_id);
\ir 04_explain.sql

REINDEX TABLE rental;
REINDEX TABLE client;
ANALYZE rental;
ANALYZE client;
\ir 04_explain.sql
```

Шаги и их смысл:

– индекс по rental.started_at – условие по дате;

– обычный индекс по client.last_name – условие по фамилии (проверка, поможет ли он при поиске по префиксу);

– индекс по client.last_name с классом операторов text_pattern_ops – предназначен для поиска LIKE 'Кузн%';

– индекс по rental.client_id – столбец внешнего ключа, по которому поездки соединяются с клиентами;

– индекс по rental.start_station_id – столбец внешнего ключа соединения со станциями;

– REINDEX таблиц rental и client и ANALYZE – перестроение индексов и обновление статистики.

7 Результаты замеров

Результаты замеров – в таблице 3 и на рисунке 5. Время – медиана пяти запусков EXPLAIN ANALYZE после одного прогревочного; стоимость – оценка корневого узла плана в условных единицах планировщика; страницы – прочитано буферов (shared hit + read). Каждый замер выполнен в новом сеансе psql, поэтому первый запуск включает прогрев служебных кэшей; при

повторных запусках в одном сеансе вариант с индексом выполняется примерно за 1 мс, а вариант без индексов – примерно за 90 мс.

Таблица 3 – Индексы, время и стоимость запроса (нарастающим итогом)

Шаг	Индексы	Время, мс	Стоимость	Страниц	Просмотр rental	Просмотр client
0	нет	101,7	9 691,5	6 655	Parallel Seq Scan	Seq Scan
1	rental(started_at)	98,6	9 691,5	6 655	Parallel Seq Scan	Seq Scan
2	+ client(last_name)	95,0	9 691,5	6 655	Parallel Seq Scan	Seq Scan
3	+ client(last_name text_pattern_ops)	94,6	9 010,4	6 051	Parallel Seq Scan	Bitmap Heap Scan
4	+ rental(client_id)	4,3	1 750,3	1 025	Index Scan (idx_rental_client_id)	Bitmap Heap Scan
5	+ rental(start_station_id)	4,3	1 750,3	1 025	Index Scan (idx_rental_client_id)	Bitmap Heap Scan
6	REINDEX и ANALYZE	4,3	1 750,1	1 025	Index Scan (idx_rental_client_id)	Bitmap Heap Scan

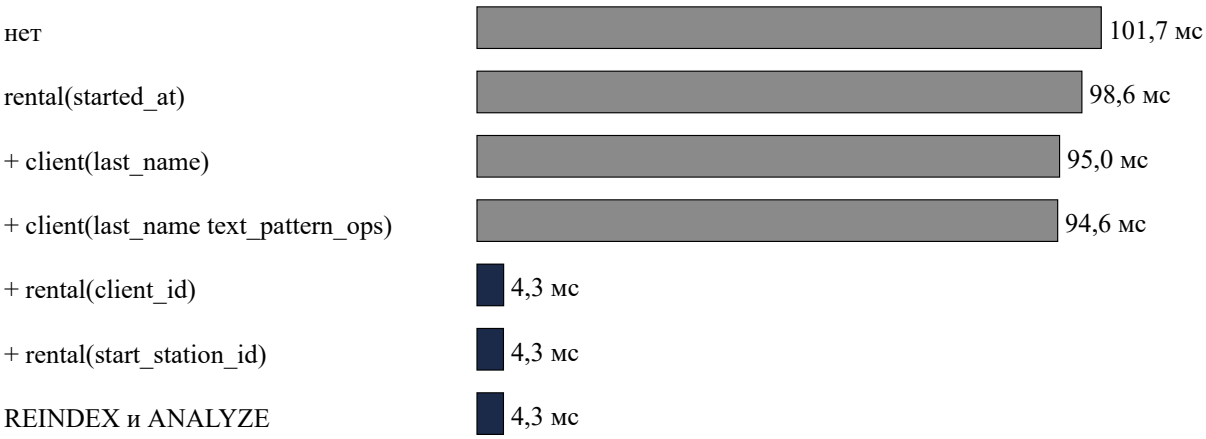


Рисунок 5 – Время выполнения запроса по шагам (медиана пяти запусков)

Что показал каждый шаг:

– индекс по started_at (8 792 КБ) не применяется: условие «с 1 января 2025 года» подходит 68,6 % строк, и при такой доле читать таблицу подряд дешевле, чем обращаться к индексу;

– обычный индекс по last_name (376 КБ) не применяется при LIKE 'Кузн%': база создана с языковым порядком сортировки (ICU ru-RU), и для префиксного поиска обычный B-tree не годится;

– индекс с text_pattern_ops (376 КБ) используется (Bitmap Index Scan) и снижает стоимость плана с 9 691,5 до 9 010,4, но время почти не меняется (94,6 мс), потому что основная работа – полный просмотр 400 000 поездок;

– индекс по rental.client_id (3 904 КБ) – решающий: поездки 31 найденного клиента берутся по индексу (в среднем две на клиента), план превращается во вложенные циклы (Nested Loop с Index Scan), время падает со 94,6 до 4,3 мс, стоимость до 1 750,3, страниц 1 025 вместо 6 655;

– индекс по start_station_id (2 840 КБ) не используется: станция присоединяется по первичному ключу, а такой индекс нужен для обратной задачи «все поездки со станции»;

– REINDEX и ANALYZE не изменили плана и времени (4,3 мс): индексы только что созданы и не раздуты, статистика свежая.

План после оптимизации (все индексы, после REINDEX и ANALYZE) показан на рисунке 6: Bitmap Heap Scan по клиентам, Nested Loop и Index Scan по idx_rental_client_id. Единичный запуск в этом выводе занял 1,8 мс, в таблице – медиана 4,3 мс.

```
PS ...\pz3-indexes-explain> npm start   # после REINDEX и ANALYZE
Limit  (cost=1790.91..1791.06 rows=58 width=67) (actual time=1.713..1.723 rows=71 loops=1)
  Buffers: shared hit=988 read=34
  ->  Sort  (cost=1790.91..1791.06 rows=58 width=67) (actual time=1.712..1.717 rows=71 loops=1)
        Sort Key: r.started_at DESC
        Sort Method: quicksort  Memory: 34kB
        Buffers: shared hit=988 read=34
        ->  Nested Loop  (cost=14.24..1789.21 rows=58 width=67) (actual time=0.259..1.672 rows=71 loops=1)
              Buffers: shared hit=988 read=34
              ->  Nested Loop  (cost=14.09..1779.63 rows=58 width=50) (actual time=0.252..1.538 rows=71 loops=1)
                    Buffers: shared hit=846 read=34
                    ->  Bitmap Heap Scan on client c  (cost=13.67..826.67 rows=24 width=31) (actual time=0.222..0.893 rows=31
loops=1)
                          Filter: ((last_name ~~ 'Кузн%'::text) AND (first_name = 'Мария'::text))
                          Rows Removed by Filter: 931
                          Heap Blocks: exact=554
                          Buffers: shared hit=554 read=3
                          ->  Bitmap Index Scan on idx_client_last_name_pattern  (cost=0.00..13.66 rows=937 width=0) (actual
time=0.140..0.140 rows=962 loops=1)
                                Index Cond: ((last_name ~>=~ 'Кузн'::text) AND (last_name ~<~ 'Кузо'::text))
                                Buffers: shared read=3
                    ->  Index Scan using idx_rental_client_id on rental r  (cost=0.42..39.68 rows=3 width=27) (actual
time=0.017..0.020 rows=2 loops=31)
                          Index Cond: (client_id = c.id)
                          Filter: ((total_cost >= '100'::numeric) AND (total_cost <= '600'::numeric) AND (started_at >= '2025-01-
01 00:00:00'::timestamp without time zone))
                          Rows Removed by Filter: 5
                          Buffers: shared hit=292 read=31
              ->  Index Scan using station_pkey on station s  (cost=0.15..0.17 rows=1 width=25) (actual time=0.001..0.001 rows=1
loops=71)
                    Index Cond: (id = r.start_station_id)
                    Buffers: shared hit=142
Planning:
  Buffers: shared hit=60 read=22
Planning Time: 1.007 ms
Execution Time: 1.767 ms
```

Рисунок 6 – План выполнения запроса после создания индексов

8 Выводы

1 Объём данных доведён до 967 801 записи (400 000 поездок), что выше требуемых 500 000; на таком объёме различие между последовательным и индексным доступом хорошо заметно.

2 Типовой запрос с соединением трёх таблиц и четырьмя условиями разных типов выполнялся без индексов 101,7 мс (последовательное чтение всей таблицы поездок); после создания индекса по rental(client_id) – 4,3 мс: ускорение в 23,4 раза, стоимость плана ниже в 5,5 раза, страниц прочитано в 6,5 раза меньше.

3 Не каждый индекс полезен: индексы по started_at, last_name (обычный) и start_station_id для этого запроса не использованы, но занимают место (около 12 МБ) и замедляют вставку и изменение данных. Эффективность индекса определяют избирательность условия (доля подходящих строк), порядок сортировки базы для текстовых условий и роль столбца в соединении.

4 Индексы по столбцам внешних ключей, используемым в соединениях, PostgreSQL автоматически не создаёт – их нужно создавать самостоятельно. Для префиксного поиска в базе с языковым порядком сортировки нужен индекс с классом операторов text_pattern_ops.

5 Перестроение индексов (REINDEX) и обновление статистики (ANALYZE) необходимы после массовой загрузки и изменений; на свежесозданных индексах они не изменили результата.